

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Mock Interview

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	S. Jaxon Rizal	Reg. No.:	192511421
Section / Batch:		Date:	24/08/26

Instructions to Students

- Attend the mock interview individually and answer all technical questions clearly.
- Explain the logic, algorithm design steps, and complexity analysis wherever required.
- Demonstrate understanding of backtracking, recursion, and combinatorial problem-solving approaches.
- Differentiate clearly between P, NP, NP-Hard, and NP-Complete problems with suitable examples.
- Maintain clear communication, confidence, and logical presentation throughout the interview.

Question Type	Focus Area	Questions
Mock Interview	Demonstrate problem-solving skills in backtracking techniques and complexity classes through technical discussion and algorithmic analysis.	Q1- Q10

SECTION A – Mock Interview

Code of Conduct

I, S. Jaxon / 192511421 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Jaxon

11. Is every NP-Complete problem also NP-hard?
Justify your answer.

Soln

Yes, every NP-Complete if it ~~also~~ NP-hard.
A problem is called NP-Complete when it satisfies two conditions: it must belong to the class NP and it must be NP-hard. NP-hard means that every problem in NP can be reduced to that problem in polynomial time. Therefore, NP-Complete problems are a subset of NP-hard problems.
However, not every NP-hard problem is NP-Complete because an NP-hard problem may not belong to NP.

Q. What is the Cook-Levin Theorem, and why is it historically significant?

Sol The Cook-Levin theorem states that the Boolean Satisfiability Problem (SAT) is NP-complete. It was proved independently by Stephen Cook and Leonid Levin. The Theorem showed that every problem in the class NP can be transformed into SAT in polynomial time. It is historically significant because it was the first major proof of an NP-complete problem and formed the foundation for studying NP-completeness. Many other problems are proved NP-complete by reducing SAT or another known NP-complete problem to them.

13. Explain how recursion supports the implementation of backtracking algorithms.

Soln Recursion plays an important role in implementing backtracking because it allows the algorithm to explore different possible choices systematically. At each recursive call, the algorithm makes a choice and moves to the next step. If the current choice leads to an invalid or unsuccessful solution, the recursive call returns to the previous state, allowing another choice to be tried. Thus, recursion automatically maintains the previous decision states. This approach is commonly used in problems such as N-Queens, Sudoku, maze solving and graph coloring.

14.

What are decision trees in Backtracking, and how are they constructed?

Soln

A decision tree in backtracking is a ~~tree~~ structure that represents all possible choices and solutions of a problem. The ~~root~~ node represents the initial state of the problem, while each branch represents a possible decision or choice. Every level of the tree generally represents one step or decision in constructing a solution. As the algorithm explores the tree, it checks whether each partial solution is valid. If a branch violates a constraint, that branch is pruned and the algorithm backtracks to explore another branch. This reduces unnecessary exploration of invalid solutions.

15. Define Constraint Satisfaction Problems (CSP) and explain how backtracking applies to them.

Soln

A Constraint Satisfaction Problem (CSP) is a problem that consists of a set of Variables, a domain of possible values for each variable, and a set of Constraints that determine which combinations of values are allowed. Backtracking solves a CSP by assigning values to variables one at a time and checking whether each assignment satisfies all relevant Constraints. If an assignment violates a Constraint, the algorithm immediately rejects that choice and goes back to the previous variable to try another value. This process continues until a complete assignment satisfying all Constraints is obtained. Examples include Sudoku, N-Queens, map coloring, and Scheduling Problems.